ENTROPY-BASED ANALYSIS OF ENCRYPTED DATA

INTRODUCTION

Entropy is a measure of uncertainty or randomness in data. In cybersecurity, it helps us study how predictable data is and understand the characteristics of encrypted information.

WHAT THIS PROJECT EXPLORES

We calculate and compare entropy values to study the statistical characteristics of encrypted data and find out how effective entropy is in deciding the security of an Encryption using shannon entropy

Why Entropy Matters for Encrypted Data

Higher entropy indicates less predictable ciphertext, making patterns harder to detect.

Randomized encryption helps hide relationships between repeated messages.

Entropy analysis helps identify predictable patterns and compare encryption outputs.

Shannon Entropy and Its Formula

$$H(X) = -\Sigma\, p(x) \log_2 p(x)$$

H(X)

Entropy

Overall unpredictability of the data, in bits

p(x)

Probability

How often symbol x shows up in the sample

log 2

Information

Bits of“surprise”carriedby one occurrence

Σ

Sum

Technical Implementation:Entropy Analysis of Encryption

Method

- Transform identical input with 8 hand-written methods (no cryptography libraries)
- Compute Shannon entropy before and after; compare via ΔH

Model

- $p(x)$ = count of byte value x ÷ total bytes (256 possible values)
- $H(X) = -\Sigma\, p(x) \log_2 p(x)$
- $\Delta H = H(Y) - H(X)$

Example: Caesar Cipher (shift = 3)

- HELLO → KHOOR; $H(X) = H(Y) \approx 1.922$ bits/char
- Substitution preserves frequencies, so entropy is unchanged

Takeaway

- Entropy measures byte-level unpredictability, not cryptographic strength

Proposed solution

A menu-driven Python application for entropy-based analysis of data, including encrypted content. Users load text, a PDF or any file, count byte frequencies, measure Shannon entropy and compare eight hand-written transformations: Caesar, Vigenère, Rail fence, XOR, one-time pad, RC4, Base64 and a random reference. Extra measures, an entropy map and an automatic verdict show how far statistics can tell structured, encoded and random-looking data apart, and where entropy fails as a security metric.

Core technical components

- Python 3 standard library only: math, sys and os
- Menu-driven console application (main.py)
- All maths in entropy_core.py, written by hand
- No cryptography libraries, no ready-made counters
- Caesar, Vigenère, Rail fence, XOR, one-time pad, RC4
- Hand-written Base64 encoder and random generator
- Min-entropy, order check, chi-square, coincidence index
- Optional: Flask web dashboard, pypdf for PDF text
- Unit testing with unittest: 37 known-answer tests

HOW IT WORKS

A Menu-Driven Application

Look at the data: byte frequencies, a 16 × 16 byte map, an entropy map and a verdict.

Experiment: the same input through 8 methods, sample size, and changing the distribution.

Understand: the limitations, a 9-step guided demo and a glossary of every term.

EXTRA MEASURES

Beyond Shannon Entropy

Min-entropy: $-\log_2$ of the most common byte's probability. The worst-case guess.

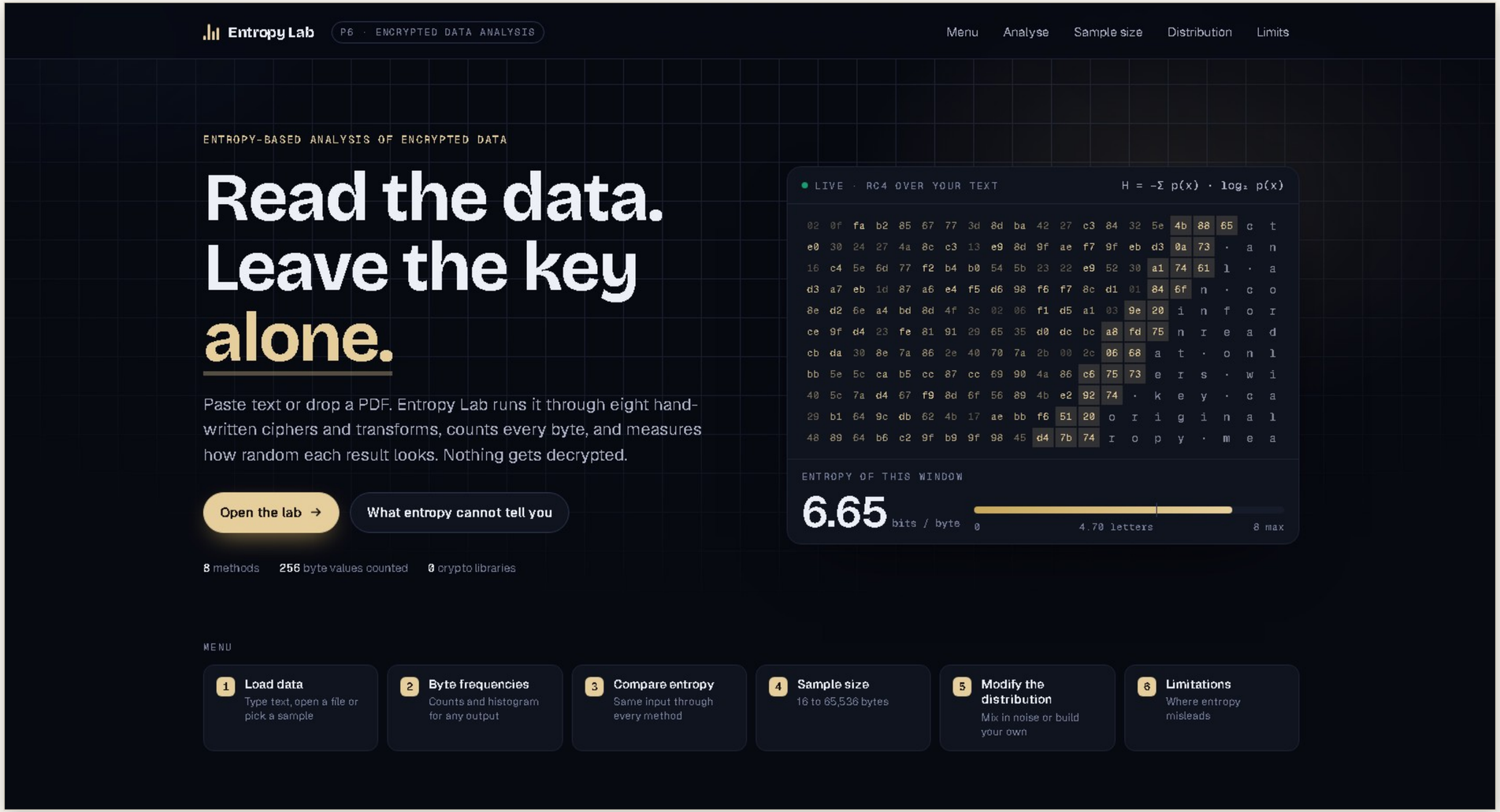
Order check: entropy of the steps between neighbouring bytes. Catches sorted and counter data.

Chi-square: are the counts as even as chance allows? Random data scores between −3 and +3.

Index of coincidence: English 0.067, random letters 0.038. Tells Caesar from Vigenère.

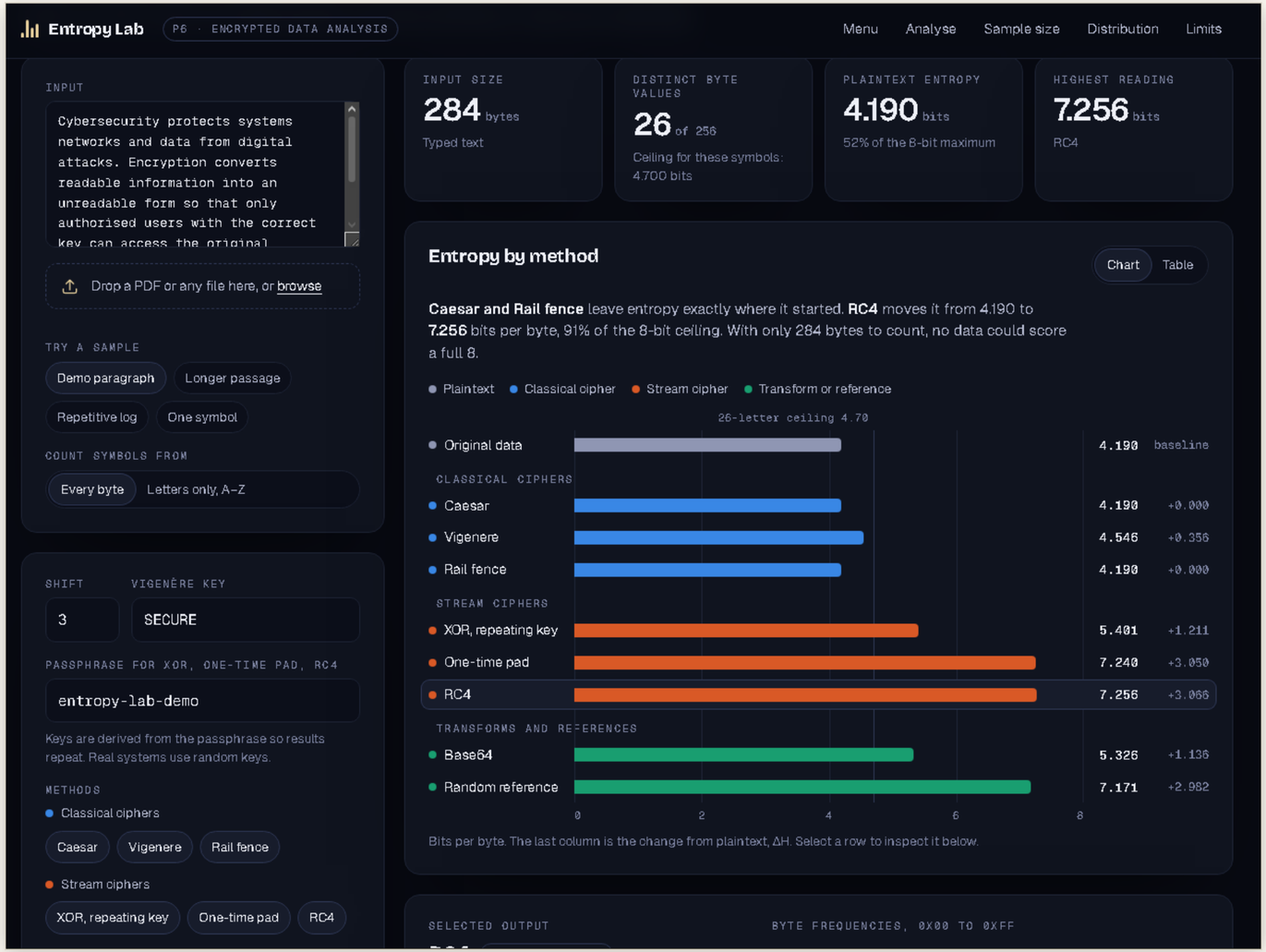
Demo: Home Screen and Menu

RC4 encrypts the text live while the window entropy updates. The numbered menu leads to every part.



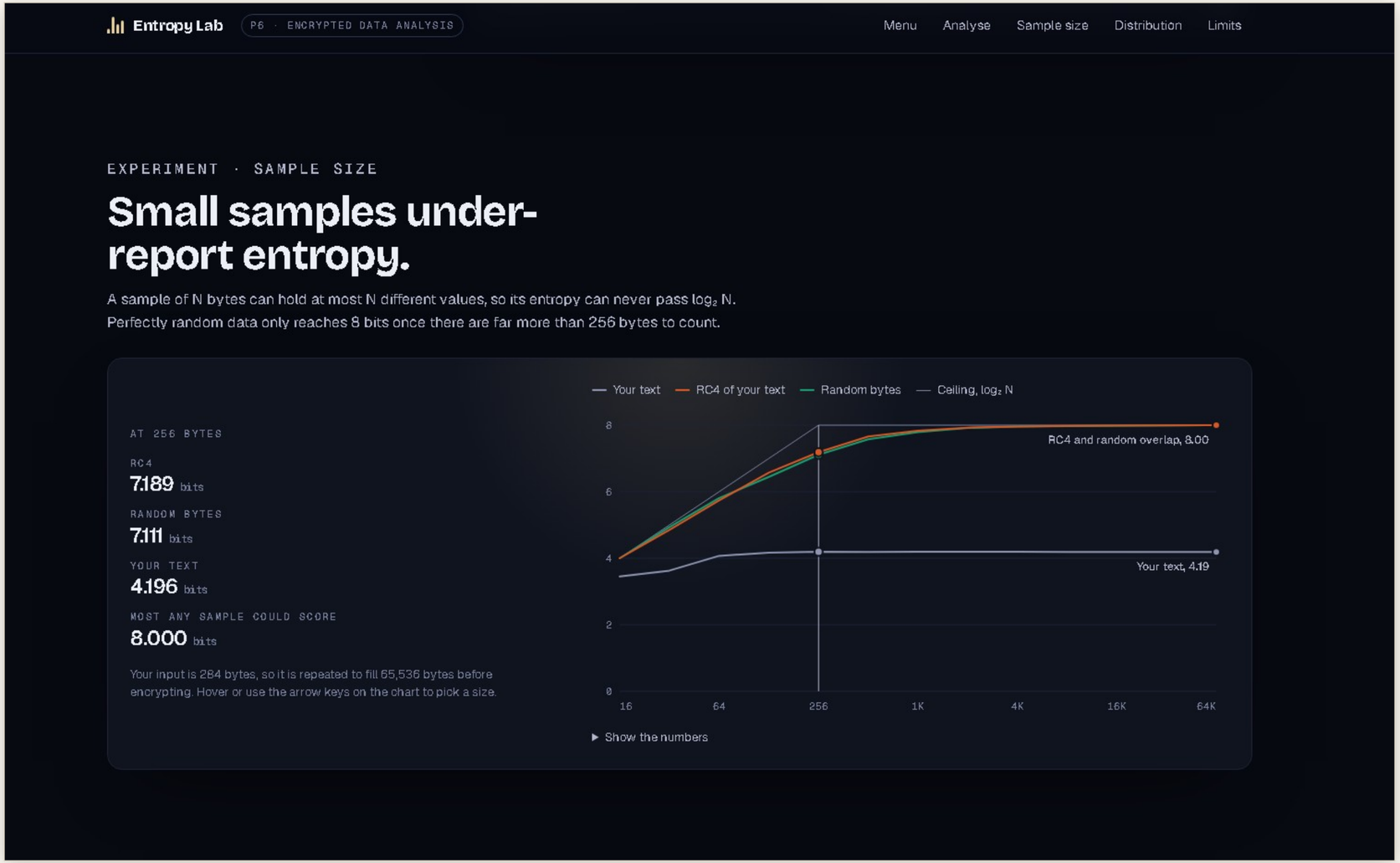
Demo: Entropy by Method

One input, eight methods. Caesar and Rail fence stay at 4.190; RC4 lifts entropy to 7.256.



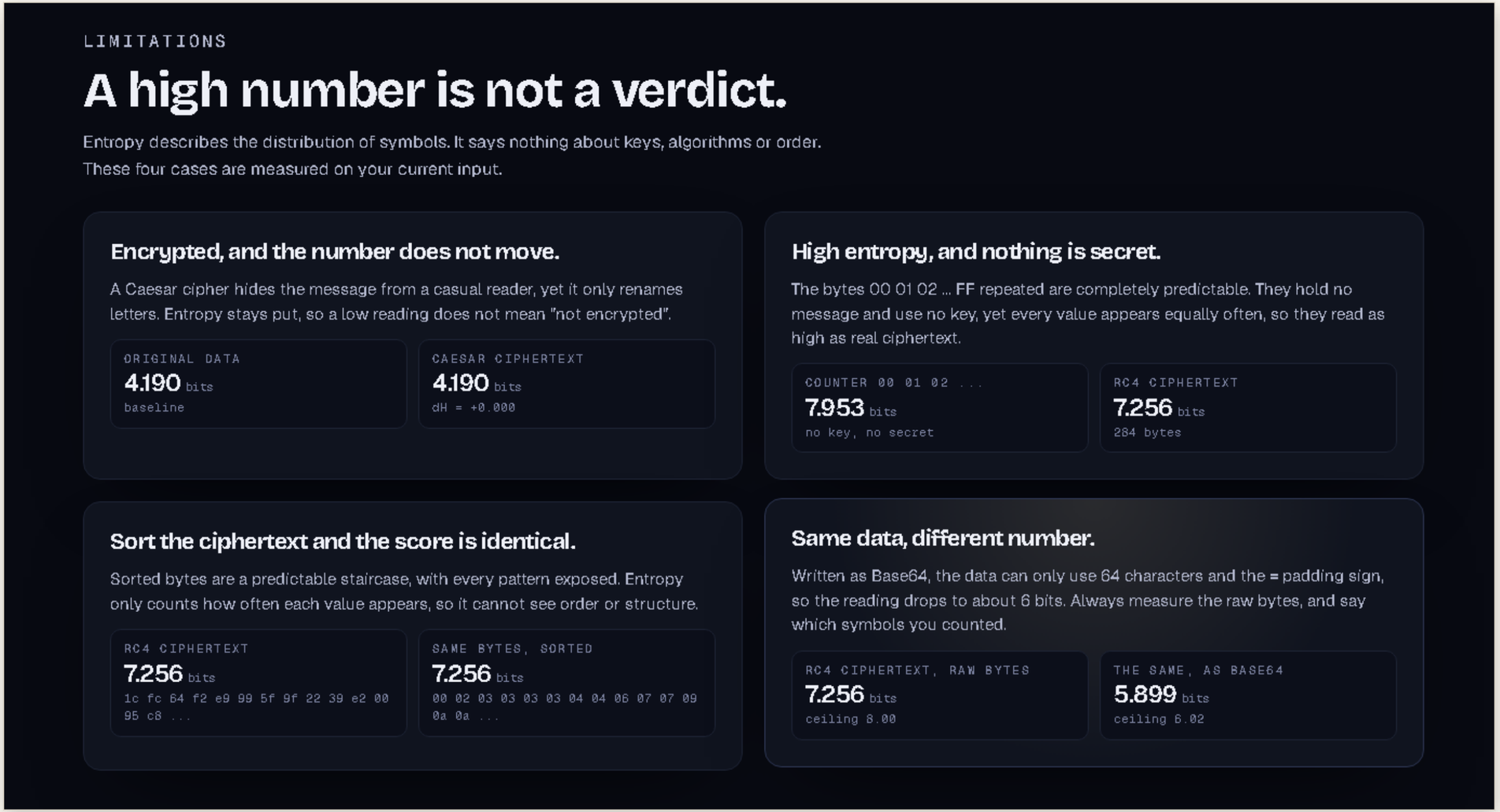
Demo: Sample-Size Experiment

At 256 bytes RC4 reads only 7.19. By 65,536 bytes RC4 and random data both reach 8.00.



Demo: Limitations, Measured

Four cases on the same input where an unchanged or high entropy reading misleads.



Limitations of Entropy Analysis

Randomness ≠ security: Weak ciphers can still produce high-entropy output.

Can't identify encryption: Compressed and random data look the same as ciphertext

Ignores structure: Patterns like repeated blocks (ECB) can hide behind uniform byte frequencies.

Sample-dependent: Small samples give noisy results, and entropy says nothing about key strength or implementation flaws.

INDIVIDUAL CONTRIBUTIONS(Primary roles)

Anupama : Mathematical Modelling

Hari:Implementaion and coding

Karthika:Algorithm design

Nisha:Presentation and technical communication

Rohit:Project coordinator and Data experimentation

Sreerag:Documentation

Vandana:Testing and Validation

Conclusion

- Entropy separates structured data from random-looking data, but cannot identify encryption
- Caesar and Rail fence hide text without changing entropy; RC4 pushes it close to 8
- Small samples read low, so compare with random data of the same size
- Extra measures and the entropy map tell more data types apart; none prove security

Questions?

